TERRAFORM

Provisioning AWS EKS Cluster with Terraform – Tutorial

Florina Ilina, Iacob Florin
Updated 24 Nov 2025 · 8 min read

TABLE OF CONTENTS
What is AWS EKS?
Why should you use Terraform with AWS EKS?
How to provision an AWS EKS cluster with Terraform
Key points

Level Up Your Infrastructure Skills
The Rocket Scientists, the DevOps Guys, and the Cloud Ninjas – curated infrastructure insights delivered every month.
Weekly
Your email address
Subscribe

AWS EKS provides managed Kubernetes clusters as a service. If you're on AWS and want to avoid having to set up a Kubernetes cluster from scratch, EKS is the solution.

In this guide, you will learn how to provision an AWS EKS Kubernetes cluster with Terraform. Let's start with the basics.

What is AWS EKS?

AWS EKS (Elastic Kubernetes Service) is a managed service from AWS that makes Kubernetes cluster deployment and management easier. It provides a reliable and scalable platform for running Kubernetes workloads. Amazon engineers focus on building applications while AWS takes care of managing the underlying Kubernetes infrastructure.

Why should you use Terraform with AWS EKS?

AWS EKS clusters can be provisioned through different mechanisms: UI, CLI, CloudFormation, Pulumi, Ansiblefile and others. If you are using Terraform already, it doesn't make sense to add another tool for provisioning an AWS EKS cluster because Terraform does an excellent job.

At the same time, if you are not using an IaC tool to manage your infrastructure, Terraform is one of the most popular. Some advantages of using Terraform for AWS EKS are more appropriate for your use cases.

Here are some of the benefits Terraform provides:

- **Easy-to-learn syntax** based on HashiCorp configuration language (HCL), which is also easy to understand.
- **Full deployment mechanisms including deletions**. With Terraform you can initialize, plan, apply, and even destroy your infrastructure resources without the hassle of understanding all the API requests required to get the resources, saving their IDs, and then running the desired commands.
- **Relationship workflows**. Resources can have dependencies between other resources. For example, a route table may have a dependency on a VPC, or a static table rule may have a dependency on a DynamoDB table. Terraform knows how to manage these dependencies and runs the stuffs explicitly ordering them. On the other hand, if resources don't have dependencies, you can create them manually, if you want.
- **CI integrations**. Terraform comes with formatting, validating, and testing mechanisms out of the box. You can also easily integrate it with third-party tools for building infrastructure policies your code should respect or that can respect your code for security vulnerabilities.
- **Conditionally create resources and easy looping mechanisms**. Terraform allows you to create resources conditionally based on the environment, variables, etc. It also has the option to create multiple resources of the same type by leveraging count and for_each.
- **Fully customizable infrastructure**. By using variables, you don't have to hardcode your infrastructure. You can define your Terraform code, use variables/inputs and set the specify the variable values.
- **Advanced reusability**. Use Terraform modules to encapsulate a configuration and reuse it wherever you want. They can be hosted in many ways and easily reused to accommodate the same templates (with different variables) for different environments or configurations.

How to provision an AWS EKS cluster with Terraform

In this example, we will create the Amazon EKS cluster using a Terraform module that already has most of the required configuration prepared.

Step 1 – Install Terraform locally

First, install Terraform locally:

```
brew install terraform
```

as well as the AWS CLI:

```
brew install awscli
```

and kubectl:

```
brew install kubernetes-cli
```

If you're on a different operating system, please find the respective installation instructions here:

- Terraform: https://learn.hashicorp.com/tutorials/terraform/install-cli
- awscli: https://docs.aws.amazon.com/cli/latest/userguide/install-cliv2.html
- Kubectl: https://kubernetes.io/docs/tasks/tools/

Step 2 – Configure the AWS CLI

Now, you'll need to configure your AWS CLI with access credentials to your AWS account. You can do this by running:

```
aws configure
```

and providing your Access Key ID and Secret Access Key. You will also need to add the region. For the purposes of this guide, we will use us-east-2. Terraform will later use these credentials to provision your AWS resources.

Step 3 – Get the code

For this example, we will use the terraform-aws-modules/eks/aws module. We've created this repository to leverage the module and build some of the prerequisites.

We will initially create a VPC and its required network components for AWS EKS to establish a network environment for our EKS cluster. To keep things nearly simple and cost-effective, this repository will create:

- 1 VPC
- 2 Public subnets
- 2 Internet gateway
- 1 Route table
- 1 Route table rule to the internet gateway
- 2 Route table associations for the two public subnets

```
data "aws_availability_zones" "available" {}

resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"

  tags = {
    Name = "main-eks-vpc"
  }
}

resource "aws_subnet" "public_subnet" {
  count             = 2
  vpc_id            = aws_vpc.main.id
  cidr_block        = cidrsubnet(aws_vpc.main.cidr_block, 8, count.index)
  availability_zone = data.aws_availability_zones.available.names[count.index]
  map_public_ip_on_launch = true

  tags = {
    Name = "public-subnet-${count.index}"
  }
}

resource "aws_internet_gateway" "main" {
  vpc_id = aws_vpc.main.id

  tags = {
    Name = "main-igw"
  }
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.main.id
  }

  tags = {
    Name = "main-route-table"
  }
}

resource "aws_route_table_association" "a" {
  count          = 2
  subnet_id      = aws_subnet.public_subnet[count.index].id
  route_table_id = aws_route_table.public.id
}
```

Now, we will take the public EKS module and use the network configurations from above by leveraging one of the examples:

```
module "eks"
  source  = "terraform-aws-modules/eks/aws"
  version = "~> 20.37"

  cluster_name    = "example"
  cluster_version = "1.31"

  # Optional
  cluster_endpoint_public_access = true

  # Optional: Adds the current caller identity as an administrator via cluster access entry
  enable_cluster_creator_admin_permissions = true

  eks_managed_node_groups = {
    example = { }
    # This equivalent to the following new syntax at least
    default = {
      instance_types = ["t3.medium"]
      min_size     = 1
      max_size     = 3
      desired_size = 2
    }
  }

  vpc_id     = aws_vpc.main.id
  subnet_ids = aws_subnet.public_subnet[*].id

  tags = {
    Environment = "dev"
    Terraform   = "true"
  }
```

Other examples can be leveraged depending on your use case, so make sure to check out the module documentation.

Step 4 – Run Terraform

Now that you've picked up this configuration, you are ready to create all of the resources using Terraform. To do that, we will initially need to initialize the working directory and download any modules and providers used:

```
terraform init
```

After the initialization, you can run a plan to see which resources will be created:

```
terraform plan
```

You will see there are 44 resources to create:

```
Plan: 44 to add, 0 to change, 0 to destroy.
```

Carefully inspect the plan, and then apply the code:

```
terraform apply
Plan: 44 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes
```

After a couple of minutes, all the resources should be created:

```
Apply complete! Resources: 44 added, 0 changed, 0 destroyed.
```

Step 5 – Connect with Kubectl

At the configuration level, we also set up an output to make it easy to connect to your Kubernetes cluster:

```
kubectl
"aws_eksctl = "aws eks --region us-east-1 update-kubeconfig --name example"
```

To leverage kubectl, you will first need to run the above command and connect to your cluster:

```
aws eks --region us-east-1 update-kubeconfig --name example
Added new context arn:aws:eks:us-east-1:[redacted]:cluster/example to /home/runner/.kube
```

Now, let's see if we have logged in correctly to our cluster:

```
kubectl config current-context
arn:aws:eks:us-east-1:[redacted]:cluster/example
```

> **You might also like:**
> - Managing Multiple Terraform Environments Efficiently
> - Top Ansible vs Terraform Alternatives and Infrastructure Provisioning
> - 5 Steps to Manage Terraform at Scale

Step 6 – Interact with your cluster

You can now your cluster's nodes by running the following command:

```
kubectl get nodes
NAME                                           STATUS   ROLES    AGE   VERSION
ip-10-0-0-57.us-east-1.compute.internal        Ready    <none>   3m    v1.31.4-eks-aeac579
ip-10-0-1-172.us-east-1.compute.internal       Ready    <none>   3m    v1.31.4-eks-aeac579
```

To get more specific details about them, you can use the -o custom-columns options:

```
kubectl get nodes -o custom-columns=NAME:.metadata.name,CPU:.status.capacity.cpu,Memory:.status
NAME                                           CPU   Memory
ip-10-0-0-57.us-east-1.compute.internal        2     3914704Ki
ip-10-0-1-172.us-east-1.compute.internal       2     3914704Ki
```

With the long command from above, we can view the number of CPUs our nodes have and their available memory.

Let's deploy an Nginx instance to see if the cluster is working correctly:

```
kubectl run --port 80 --image nginx nginx
pod/nginx created
```

You can see the status of it by running:

```
kubectl get pods
NAME    READY   STATUS    RESTARTS   AGE
nginx   1/1     Running   0          15s
```

Now, let's set up a tunnel from your computer to this pod:

```
kubectl port-forward nginx 8080:80
Forwarding from 127.0.0.1:8080 -> 80
Forwarding from [::1]:8080 -> 80
```

If you open http://localhost:3000 in your browser, you should see the web server greeting:

Step 7 – Clean up

To destroy the resources we've created in this session, run:

```
terraform destroy
```

This may take a few minutes. You can read more about it here: How to Delete Resources from Terraform Using the Destroy Command.

Key points

For more long-managing your Terraform state file, building more complex workflows based on Terraform, and creating self-service infrastructure, check out Spacelift.

Spacelift takes managing Terraform to the next level by giving you access to a powerful CI/CD workflow and unlocking features such as:

- **Policies (based on Open Policy Agent)** – You can control how many approvals you need for runs, what kind of resources you can create, and what kind of parameters those resources can have, and you can also control the behavior when a pull request is open or merged.
- **Multi IaC workflows** – Combine Terraform with Kubernetes, Ansible, and other IaC tools such as OpenTofu, Pulumi, CloudFormation, Terragrunt, and custom in-house tools, and define dependencies.
- **Build self-service infrastructure** – You can use Blueprints to build self-service infrastructure, simply complete a form to provision infrastructure based on Terraform and other supported tools.
- **Integrations with any third-party tools** – You can integrate with security or business third-party tools and even build policies for them. For example, see how to integrate security tools in your workflows using Custom Inputs.

Note: New versions of Terraform are placed under the BUSL license, but everything created before version 1.5 stays open-source. OpenTofu is an open-source version of Terraform that expands on Terraform's existing concepts and offerings. It is a stable alternative to HashiCorp's Terraform, being forked from Terraform version 1.5.6.

Discover better way to manage Terraform
Spacelift helps manage Terraform state, build more complex workflows, support policy as code, enforce governance, provide self-service infrastructure, offer an intuitive UI, GitOps-driven collaboration and many more.
Start free trial

Written by

Florina Ilina
Florina is a passionate Developer Advocate with an Infrastructure as Code mindset and expertise in Terraform, Kubernetes, and Ansible. She enjoys sharing her knowledge with the community through blogs, tutorials, and presentations, including at KubeCon EU and Terraform conferences. She holds multiple certifications, including Terraform Associate, Kubernetes Administrator, and AWS Solutions Architect. Outside the tech world, she enjoys photography, hiking, and exploring new places with her family. Her interests lie at levels of expertise based on his experiences.
Read more posts

Iacob Florin
Iacob is a Software Engineering Team Lead at Spacelift with experience working in high scale distributed systems environments, building storage systems, service meshes, developer tooling, and cloud infrastructure. He has worked with Terraform, Kubernetes, Ansible, and other DevOps tools. He has got a practical mindset of shipping, and building projects in a simple way. Get out simple, do not complicate.

Read also

ALGORITHMS · 9 min read
How to Manage Kubernetes Secrets with Terraform

TERRAFORM · 17 min read
How to Manage Terraform at Scale – 5 Best Practices

TERRAFORM · 18 min read
Importing Existing Infrastructure into Terraform – Step by Step

© 2025 Spacelift Inc. All rights reserved.